

**LAPORAN PRAKTIKUM
ALGORITMA PEMROGRAMAN 2
MODUL 12 – SEARCHING**



**HAIDAR RAFI ARSHIA
109082530022
KELAS S1IF-13-06**

**PROGRAM STUDI TEKNIK INFORMATIKA
FAKULTAS INFORMATIKA
TELKOM UNIVERSITY PURWOKERTO
2026**

A. Dasar Teori

Algoritma pencarian merupakan salah satu proses fundamental dalam pemrograman yang bertujuan untuk menemukan lokasi atau keberadaan suatu data spesifik di dalam sekumpulan data. Berbeda dengan pencarian nilai ekstrim yang hasilnya selalu pasti ditemukan, pencarian data spesifik seperti mencari data mahasiswa berdasarkan NIM atau nama memiliki kemungkinan bahwa data yang dicari tidak ada di dalam kumpulan data tersebut. Oleh karena itu, algoritma pencarian modern tidak hanya dirancang untuk mengembalikan status kebenaran (apakah data ditemukan atau tidak), tetapi juga sering kali dimodifikasi untuk menghasilkan indeks posisi dari data tersebut agar bisa diolah lebih lanjut.

Salah satu metode yang paling sederhana adalah *Sequential Search* atau pencarian sekuensial, di mana proses pemeriksaan dilakukan secara linier dengan mengecek elemen array satu per satu dari data pertama hingga data terakhir. Ciri khas utama dari algoritma ini adalah proses pencarian akan langsung dihentikan seketika data yang dicari telah ditemukan, meskipun masih terdapat sisa elemen array yang belum diperiksa. Karena tidak bergantung pada susunan data, metode sekuensial ini sangat fleksibel dan dapat digunakan pada kumpulan data yang acak atau tidak terurut.

Metode kedua adalah *Binary Search* atau pencarian biner, yang menawarkan efisiensi waktu jauh lebih tinggi namun memiliki syarat mutlak, yaitu data di dalam array harus sudah dalam keadaan terurut, baik secara membesar (*ascending*) maupun mengecil (*descending*). Algoritma ini bekerja dengan cara membagi ruang pencarian menjadi dua bagian secara berulang dengan memanfaatkan nilai tengah (*median*) sebagai acuan. Jika nilai yang dicari lebih besar atau lebih kecil dari nilai tengah, rentang pencarian akan digeser ke arah yang sesuai dan mengabaikan setengah data lainnya, sehingga proses pencarian menjadi sangat cepat. Penerapan kedua metode ini juga dapat diperluas untuk tipe data terstruktur seperti *struct*, dengan catatan kategori urutan data pada array harus selaras dengan kolom (*field*) yang dijadikan dasar pencarian.

B. Guided

1. Guided1.go

```
1 package main
2
3 import "fmt"
4
5 type arrData [5]string //tipe data alias
6
7 func seqSearch(arr arrData, binatangCari string) int {
8     var found int = -1
9     for i := 0; i < len(arr); i++ {
10         if arr[i] == binatangCari {
11             found = i
12             break
13         }
14     }
15     return found
16 }
17
18 func main() {
19     var arrBinatang arrData //array tipe string
20
21     for i := 0; i < len(arrBinatang); i++ {
22         fmt.Printf("Masukkan data binatang indeks ke-%d : ", i)
23         fmt.Scan(&arrBinatang[i])
24     }
25     fmt.Println()
26
27     var binatangCari string
28     fmt.Print("Masukkan nama binatang yang mau dicari : ")
29     fmt.Scan(&binatangCari)
30
31     var idxCari int
32     idxCari = seqSearch(arrBinatang, binatangCari)
33
34     if idxCari > -1 {
35         fmt.Printf("Data %s ditemukan pada indeks ke-%d!", binatangCari, idxCari)
36     } else if idxCari == -1 {
37         fmt.Printf("Data %s tidak ditemukan!", binatangCari)
38     }
39
40 }
```

Output:

```
● PS D:\SEMESTER 2\ALPRO\PRAKTIKUM\109082530022_Haidar-Rafi-Arshia> go run  
go"  
Masukkan data binatang indeks ke-0 : Kucing  
Masukkan data binatang indeks ke-1 : Anjing  
Masukkan data binatang indeks ke-2 : Kelinci  
Masukkan data binatang indeks ke-3 : Singa  
Masukkan data binatang indeks ke-4 : Harimau  
  
Masukkan nama binatang yang mau dicari : Kelinci  
Data Kelinci ditemukan pada indeks ke-2!  
○ PS D:\SEMESTER 2\ALPRO\PRAKTIKUM\109082530022_Haidar-Rafi-Arshia> █
```

Penjelasan:

Program ini mengimplementasikan algoritma pencarian sekuensial (*pencarian sekuensial*) untuk menemukan posisi suatu data string di dalam array yang disusun dengan ukuran statistik. Proses diawali dengan mengisi seluruh elemen array menggunakan perulangan, kemudian program melakukan penelusuran satu per satu dari indeks awal hingga akhir untuk mencocokkan setiap elemen dengan kata kunci binatang yang dicari. Jika data ditemukan, perulangan akan langsung dihentikan demi efisiensi dan program akan mengembalikan nilai indeks spesifik tempat data tersebut berada, sedangkan jika seluruh array telah diperiksa dan tidak ada kecocokan, fungsi akan mengembalikan nilai penanda -1. Hasil akhir pencarian ini kemudian divalidasi melalui percabangan kondisi untuk memberikan informasi yang informatif kepada pengguna, apakah lokasi data berhasil diidentifikasi atau dinyatakan tidak ditemukan di dalam sistem.

2. Guided2.go



```
1  guided binary search
2  package main
3
4  import "fmt"
5
6  // fungsi binary search
7  func binarySearch(data []int, x int) int {
8
9      // batas kiri array
10     kiri := 0
11
12     // batas kanan array
13     kanan := len(data) - 1
14
15     // perulangan selama kiri <= kanan
16     for kiri <= kanan {
17
18         // mencari index tengah
19         tengah := (kiri + kanan) / 2
20
21         // jika data ditemukan
22         if data[tengah] == x {
23             return tengah
24
25         // jika x lebih besar, cari ke kanan
26         } else if data[tengah] < x {
27             kiri = tengah + 1
28
29         // jika x lebih kecil, cari ke kiri
30         } else {
31             kanan = tengah - 1
32         }
33     }
34
35     // jika data tidak ditemukan
36     return -1
37 }
```

```
38
39 func main() {
40
41     var n int
42     var cari int
43
44     // input jumlah data
45     fmt.Print("Jumlah data: ")
46     fmt.Scan(&n)
47
48     // membuat array sesuai jumlah data
49     data := make([]int, n)
50
51     // input isi array
52     fmt.Println("Masukkan data terurut:")
53     for i := 0; i < n; i++ {
54         fmt.Scan(&data[i])
55     }
56
57     // input angka yang dicari
58     fmt.Print("Angka yang dicari: ")
59     fmt.Scan(&cari)
60
61     // memanggil fungsi binary search
62     hasil := binarySearch(data, cari)
63
64     // output hasil pencarian
65     if hasil != -1 {
66         fmt.Println("Data ditemukan di index:", hasil)
67     } else {
68         fmt.Println("Data tidak ditemukan")
69     }
70 }
```

Output :

```
PS D:\SEMESTER 2\ALPRO\PRAKTIKUM\109082530022_Haidar-Rafi-Arshia> go run
go"
Jumlah data: 5
Masukkan data terurut:
2
5
8
12
20
Angka yang dicari: 12
Data ditemukan di index: 3
PS D:\SEMESTER 2\ALPRO\PRAKTIKUM\109082530022_Haidar-Rafi-Arshia> |
```

Penjelasan:

Program ini mengimplementasikan pencarian nilai ekstrim pada **tipe data terstruktur (struct)** untuk mengelola data siswa, di mana perbandingan dilakukan berdasarkan lapangan IPK namun tetap mempertahankan keutuhan seluruh identitas siswa tersebut. Berikut konsep pada modul 10.3, fungsi IPK Terbesar dan IPK Terkecil menggunakan elemen pertama array sebagai nilai acuan untuk divalidasi secara sekuensial terhadap data siswa lainnya. Dengan mengembalikan variabel bertipe struct dan nilai **indeks**, program dapat secara efisien menampilkan informasi lengkap (nama dan NIM) siswa dengan IPK tertinggi atau terendah tanpa kehilangan jejak lokasi di dalam array.

3. Guided3.go

```
1 package main
2
3 import "fmt"
4
5 type mahasiswa struct {
6     nama string
7     nim int
8     IPK float64
9 }
10
11 type arrMahasiswa [5]mahasiswa
12
13 func binSearchStruct(arrData arrMahasiswa, nimDicari int) int {
14     var found int = -1
15     var med int
16     var kr int = 0
17     var kn int = len(arrData) - 1
18     for kr <= kn && found == -1 {
19         med = (kr + kn) / 2
20         if nimDicari < arrData[med].nim {
21             kn = med - 1
22         } else if nimDicari > arrData[med].nim {
23             kr = med + 1
24         } else {
25             found = med
26         }
27     }
28     return found
29 }
```

```

31 func seqSearchStruct(arrData arrMahasiswa, nimDicari int) int {
32     var found int = -1
33     for i := 0; i < len(arrData); i++ {
34         if arrData[i].nim == nimDicari {
35             found = i
36             break
37         }
38     }
39     return found
40 }
41
42 func main() {
43     var mahasiswa06 arrMahasiswa
44     var nimDicari int
45
46     fmt.Println("--- MASUKKAN DATA NIM MAHASISWA SECARA TERURUT ---")
47     for i := 0; i < len(mahasiswa06); i++ {
48         fmt.Printf("=== Masukkan data mahasiswa 06 pada indeks ke-%d ===\n", i)
49         fmt.Print("Masukkan nama : ")
50         fmt.Scan(&mahasiswa06[i].nama)
51         fmt.Print("Masukkan NIM : ")
52         fmt.Scan(&mahasiswa06[i].nim)
53         fmt.Print("Masukkan IPK : ")
54         fmt.Scan(&mahasiswa06[i].IPK)
55         fmt.Println("-----")
56     }
57     fmt.Println()
58
59     fmt.Print("Masukkan NIM mahasiswa yang ingin dicari : ")
60     fmt.Scan(&nimDicari)
61     fmt.Println()
62
63     var idxHasilCariSeqSearch int
64     idxHasilCariSeqSearch = seqSearchStruct(mahasiswa06, nimDicari)
65
66     fmt.Println("== HASIL SEQUENTIAL SEARCH ==")
67     if idxHasilCariSeqSearch > -1 {
68         fmt.Printf("Data NIM mahasiswa %d ditemukan pada array di indeks ke-%d!\n", nimDicari, idxHasilCariSeqSearch)
69         fmt.Println("Berikut Detail Datanya : ")
70         fmt.Printf("Nama : %s\n", mahasiswa06[idxHasilCariSeqSearch].nama)
71         fmt.Printf("NIM : %d\n", mahasiswa06[idxHasilCariSeqSearch].nim)
72         fmt.Printf("IPK : %.2f\n", mahasiswa06[idxHasilCariSeqSearch].IPK)
73     } else if idxHasilCariSeqSearch == -1 {
74         fmt.Printf("Data NIM mahasiswa %d tidak ditemukan pada array!", nimDicari)
75     }
76     fmt.Println()
77
78     var idxHasilCariBinSearch int
79     idxHasilCariBinSearch = binSearchStruct(mahasiswa06, nimDicari)
80
81     fmt.Println("== HASIL BINARY SEARCH ==")
82     if idxHasilCariBinSearch > -1 {
83         fmt.Printf("Data NIM mahasiswa %d ditemukan pada array di indeks ke-%d!\n", nimDicari, idxHasilCariBinSearch)
84         fmt.Println("Berikut Detail Datanya : ")
85         fmt.Printf("Nama : %s\n", mahasiswa06[idxHasilCariBinSearch].nama)
86         fmt.Printf("NIM : %d\n", mahasiswa06[idxHasilCariBinSearch].nim)
87         fmt.Printf("IPK : %.2f\n", mahasiswa06[idxHasilCariBinSearch].IPK)
88     } else if idxHasilCariBinSearch == -1 {
89         fmt.Printf("Data NIM mahasiswa %d tidak ditemukan pada array!", nimDicari)
90     }
91 }

```


Output:

```
● PS D:\SEMESTER 2\ALPRO\PRAKTIKUM\109082530022_Haidar-Rafi-Arshia> go run
go"
--- MASUKKAN DATA NIM MAHASISWA SECARA TERURUT ---
=== Masukkan data mahasiswa 06 pada indeks ke-0 ===
Masukkan nama : Haidar
Masukkan NIM : 1001
Masukkan IPK : 3.5
-----
=== Masukkan data mahasiswa 06 pada indeks ke-1 ===
Masukkan nama : Budi
Masukkan NIM : 1002
Masukkan IPK : 3.75
-----
=== Masukkan data mahasiswa 06 pada indeks ke-2 ===
Masukkan nama : Citra
Masukkan NIM : 1003
Masukkan IPK : 3.85
-----
=== Masukkan data mahasiswa 06 pada indeks ke-3 ===
Masukkan nama : Dimas
Masukkan NIM : 1004
Masukkan IPK : 3.2
-----
=== Masukkan data mahasiswa 06 pada indeks ke-4 ===
Masukkan nama : Elsa
Masukkan NIM : 1005
Masukkan IPK : 3.9
-----

Masukkan NIM mahasiswa yang ingin dicari : 1003

== HASIL SEQUENTIAL SEARCH ==
Data NIM mahasiswa 1003 ditemukan pada array di indeks ke-2!
Berikut Detail Datanya :
Nama : Citra
NIM : 1003
IPK : 3.85

== HASIL BINARY SEARCH ==
Data NIM mahasiswa 1003 ditemukan pada array di indeks ke-2!
Berikut Detail Datanya :
Nama : Citra
NIM : 1003
IPK : 3.85
○ PS D:\SEMESTER 2\ALPRO\PRAKTIKUM\109082530022_Haidar-Rafi-Arshia> |
```

Penjelasan:

Program ini mengimplementasikan dua metode pencarian yang berbeda, yaitu pencarian sekuensial (*sequential search*) dan pencarian biner (*binary search*), untuk menemukan data siswa di dalam array bertipe *struct* . Setelah pengguna menginputkan data lima mahasiswa yang wajib terurut berdasarkan NIM dan satu NIM target yang ingin dicari, program akan menjalankan kedua fungsi pencarian tersebut secara bergantian. Fungsi pertama memproses data dengan memeriksa setiap elemen dari indeks awal hingga akhir secara linier, sementara fungsi kedua membagi ruang pencarian menjadi dua bagian secara berulang dengan memanfaatkan titik tengah untuk menyebarkan jangkauan secara efisien. Apabila data ditemukan oleh masing-masing fungsi, program akan mengembalikan nilai indeksnya untuk kemudian menampilkan informasi lengkap siswa berupa nama, NIM, dan posisi IPK secara komprehensif, namun jika kunci pencarian tidak ditemukan di seluruh elemen array, kedua fungsi akan mengembalikan nilai penanda -1 yang memicu blok kondisi untuk menginfokan bahwa data tersebut tidak ada di dalam sistem.

C. Unguided

1. Soal Unguided 1

PenghitunganSuaraPemilihanKetuaRT.go

Jawaban :

```
1  package main
2
3  import (
4      "fmt"
5  )
6
7  func main() {
8      var suara, suaraMasuk, suaraSah int
9      var hitungSuara [21]int
10
11     for {
12         fmt.Scan(&suara)
13         if suara == 0 {
14             break
15         }
16         suaraMasuk++
17
18         if suara >= 1 && suara <= 20 {
19             suaraSah++
20             hitungSuara[suara]++
21         }
22     }
23
24     fmt.Printf("Suara masuk: %d\n", suaraMasuk)
25     fmt.Printf("Suara sah: %d\n", suaraSah)
26
27     for i := 1; i <= 20; i++ {
28         if hitungSuara[i] > 0 {
29             fmt.Printf("%d:%d\n", i, hitungSuara[i])
30         }
31     }
32 }
```

Output :

```
● PS D:\SEMESTER 2\ALPRO\PRAKTIKUM\109082530022_Haidar-Rafi-Arshia> go run  
nghitunganSuaraPemilihanKetuaRT.go"  
 7 19 3 2 78 3 1 -3 18 19 0  
Suara masuk: 10  
Suara sah: 8  
1:1  
2:1  
3:2  
7:1  
18:1  
19:2  
○ PS D:\SEMESTER 2\ALPRO\PRAKTIKUM\109082530022_Haidar-Rafi-Arshia> █
```

Penjelasan :

Program ini dirancang untuk membaca, memvalidasi, dan menghitung perolehan suara dalam pemilihan ketua RT yang diikuti oleh 20 calon. Pengguna memasukkan data dalam satu baris berupa deretan nomor calon, di mana proses input akan otomatis berhenti ketika mendeteksi angka 0. Program secara berulang (menggunakan perulangan *looping*) membaca setiap suara, memeriksa validitasnya (apakah berada pada rentang angka 1 hingga 20), dan memperbarui variabel pencatat jumlah total suara masuk serta suara yang sah. Distribusi suara disimpan di dalam sebuah array frekuensi berukuran tetap. Di akhir proses, program memproses isi array secara sekuensial dari nomor urut terkecil hingga terbesar untuk menampilkan total suara masuk, total suara sah, beserta detail nomor calon dan jumlah suara yang berhasil mencapai raih mereka secara terperinci.

2. Soal Unguided 2

PenentuanKetudahanWakilKetuaRTTerpilih.go

Jawaban :



```
1  package main
2
3  import (
4      "fmt"
5  )
6
7  func main() {
8      var suara, suaraMasuk, suaraSah int
9      var hitungSuara [21]int
10
11     for {
12         fmt.Scan(&suara)
13         if suara == 0 {
14             break
15         }
16         suaraMasuk++
17
18         if suara >= 1 && suara <= 20 {
19             suaraSah++
20             hitungSuara[suara]++
21         }
22     }
23
24     fmt.Printf("Suara masuk: %d\n", suaraMasuk)
25     fmt.Printf("Suara sah: %d\n", suaraSah)
26
27     ketua := -1
28     wakil := -1
29     maks1 := -1
30     maks2 := -1
```

```

31
32     kecil (diakomodasi oleh '<' pada iterasi ascending)
33     for i := 1; i <= 20; i++ {
34         if hitungSuara[i] > maks1 {
35             maks2 = maks1
36             wakil = ketua
37             maks1 = hitungSuara[i]
38             ketua = i
39         } else if hitungSuara[i] > maks2 {
40             maks2 = hitungSuara[i]
41             wakil = i
42         }
43     }
44
45     fmt.Printf("Ketua RT: %d\n", ketua)
46     fmt.Printf("Wakil ketua: %d\n", wakil)
47 }

```

Output :

```

PS D:\SEMESTER 2\ALPRO\PRAKTIKUM\109082530022_Haidar-Rafi-Arshia> go run
nentuanKetuaWakilKetuaRTTerpilih.go
7 19 3 2 78 3 1 -3 18 19 0
Suara masuk: 10
Suara sah: 8
Ketua RT: 3
Wakil ketua: 19
PS D:\SEMESTER 2\ALPRO\PRAKTIKUM\109082530022_Haidar-Rafi-Arshia>

```

Penjelasan :

Program ini merupakan pengembangan dari sistem perhitungan suara sebelumnya, dengan fokus utama mencari pasangan Ketua dan Wakil Ketua RT berdasarkan perolehan suara terbanyak. Mekanisme pembacaan data dan validasi suara sah (rentang 1-20) serta tanda berhenti angka 0 tetap dipertahankan. Modifikasi krusial terletak pada algoritma penentuan pemenang setelah semua data dibaca, di mana program melakukan penelusuran linier (*sequential scan*) pada array frekuensi untuk menyaring dua nilai ekstrem tertinggi. Sesuai ketentuan, jika terdapat calon yang mengantongi jumlah suara sama banyak, program akan otomatis memprioritaskan calon dengan nomor urut yang paling kecil karena iterasi array bergerak maju dari indeks 1 hingga 20. Output akhir program akan menginformasikan jumlah statistik suara serta menyajikan nomor peserta yang resmi terpilih menjadi Ketua dan Wakil Ketua RT.

3. Soal Unguided 3

PencarianIndeksDataTerurut.go

Jawab:



```
1  package main
2
3  import "fmt"
4
5  const NMAX = 1000000
6
7  var data [NMAX]int
8
9  func main() {
10     var n, k int
11
12     fmt.Scan(&n, &k)
13
14     isiArray(n)
15     pos := posisi(n, k)
16
17     if pos != -1 {
18         fmt.Println(pos)
19     } else {
20         fmt.Println("TIDAK ADA")
21     }
```

```
22 }
23
24 func isiArray(n int) {
25     for i := 0; i < n; i++ {
26         fmt.Scan(&data[i])
27     }
28 }
29
30 func posisi(n, k int) int {
31     kr := 0
32     kn := n - 1
33     found := -1
34
35     for kr <= kn && found == -1 {
36         med := (kr + kn) / 2
37         if data[med] == k {
38             found = med
39         } else if data[med] < k {
40             kr = med + 1
41         } else {
42             kn = med - 1
43         }
44     }
45     return found
46 }
47
```


Output:

```
● PS D:\SEMESTER 2\ALPRO\PRAKTIKUM\109082530022_Haidar-Rafi-Arshia> go run
mpCodeRunnerFile.go"
12 534
1 3 8 16 32 123 323 323 534 543 823 999
8
● PS D:\SEMESTER 2\ALPRO\PRAKTIKUM\109082530022_Haidar-Rafi-Arshia> go run
ncarianIndeksDataTerurut.go"
12 535
1 3 8 16 32 123 323 323 534 543 823 999
TIDAK ADA
○ PS D:\SEMESTER 2\ALPRO\PRAKTIKUM\109082530022_Haidar-Rafi-Arshia> |
```

Penjelasan :

Menggunakan subprogram untuk mencari angka\$\$dalam\$\$data (hingga 1.000.000 data). Karena data sudah terurut membesar, fungsi mencari menggunakan **Binary Search** dengan membagi dua wilayah data lewat nilai tengah secara berulang agar proses menjadi sangat cepat.

D. Kesimpulan

Berdasarkan **Modul 12** , kita dapat menyimpulkan bahwa mencari nilai ekstrim sebenarnya dilakukan satu per satu dengan membandingkan setiap elemen terhadap nilai acuan yang diperbarui terus-menerus. Cara ini tidak hanya berguna untuk mencari satu nilai pada tipe data dasar, tetapi juga sangat penting dalam mengenali objek pada tipe data terstruktur, seperti *struct* , melalui indeks penyimpanan. Agar algoritma ini berhasil, kita harus memastikan inisialisasi yang tepat pada elemen pertama dan melakukan validasi secara menyeluruh hingga akhir kumpulan data.